

Decisions + Loops, Together

Week 2 — Composing what we already know: conditional accumulation, running-best tracking, nested loops, validation, and search.

What We'll Combine This Week

1

Conditional accumulation

A loop where an if decides whether to update a running total or count.

2

Running best (max & min)

Tracking more than one “best so far” in the same pass.

3

A loop inside a loop

A genuinely new shape — the inner loop repeats completely for every pass of the outer one.

4

Input validation

Looping back to an input step, not a decision, until the value is acceptable.

5

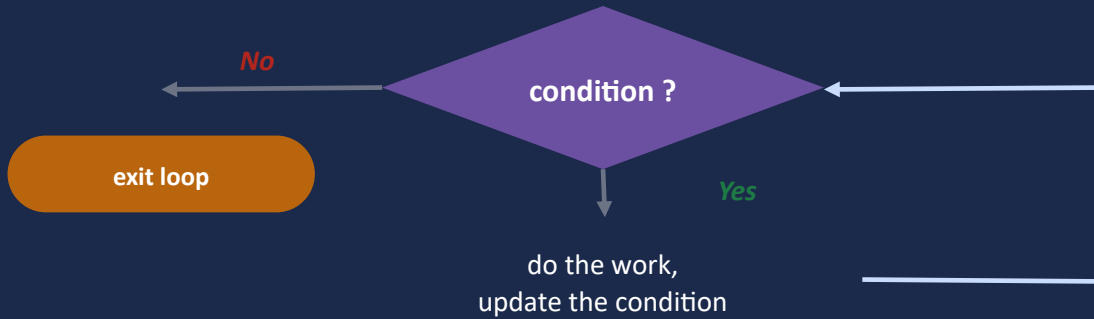
Search with early stop

A loop that can end two different ways — found, or ran out of items to check.

STILL NO JAVA SYNTAX

Every idea this week stays in flowcharts, pseudocode, and trace tables — the same toolkit as last week, aimed at harder problems.

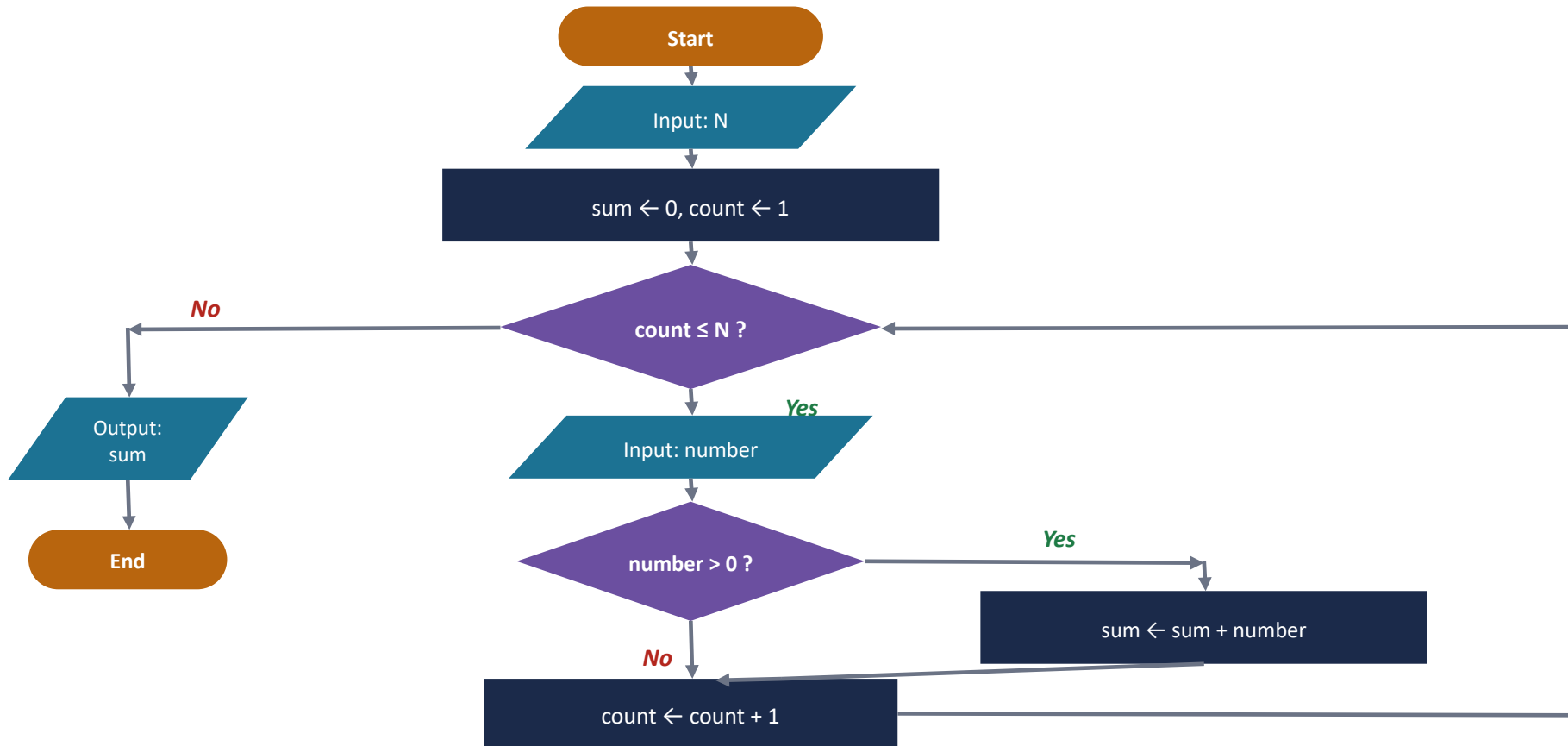
One Idea We're Building On



A loop is a decision with an arrow pointing back to it.

Everything new this week is still built from exactly four symbols: Terminal, Process, Decision, Input/Output. We're just combining them in less obvious ways — a decision inside a loop, two decisions side by side, a loop inside a loop.

Sum Only the Positive Numbers



THE PATTERN

Every pass reads one number. The if decides whether that pass changes sum — count always updates, sum only sometimes does.

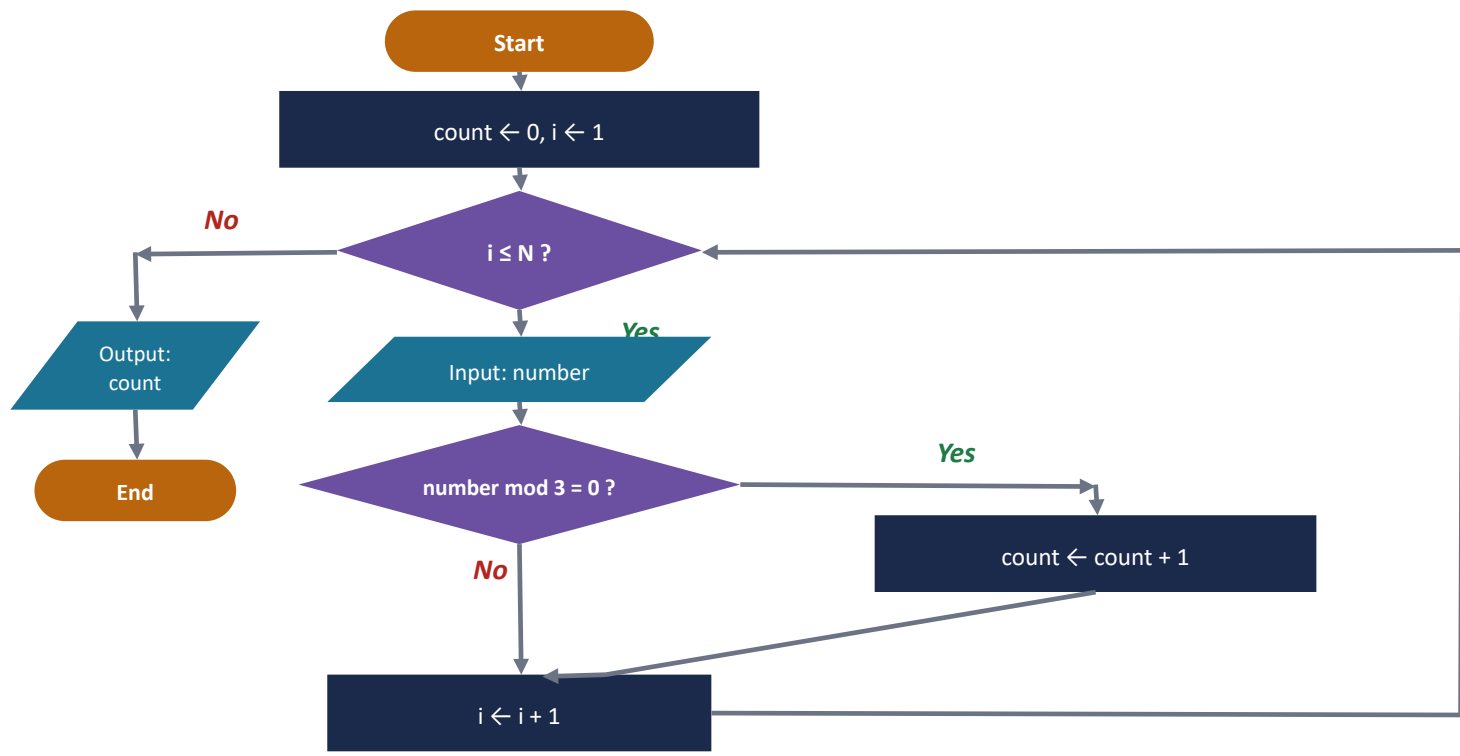
Tracing “Sum Positives” by Hand

Sample run with $N = 5$, inputs: 3, -2, 7, -1, 4.

count	number	number > 0 ?	sum
1	3	Yes	$0 + 3 = 3$
2	-2	No	3
3	7	Yes	$3 + 7 = 10$
4	-1	No	10
5	4	Yes	$10 + 4 = 14$
6	—	exit (count > N)	output 14

Notice: count climbs every single pass; sum only climbs on the passes where the number was positive.

A Second Shape: Counting Instead of Summing

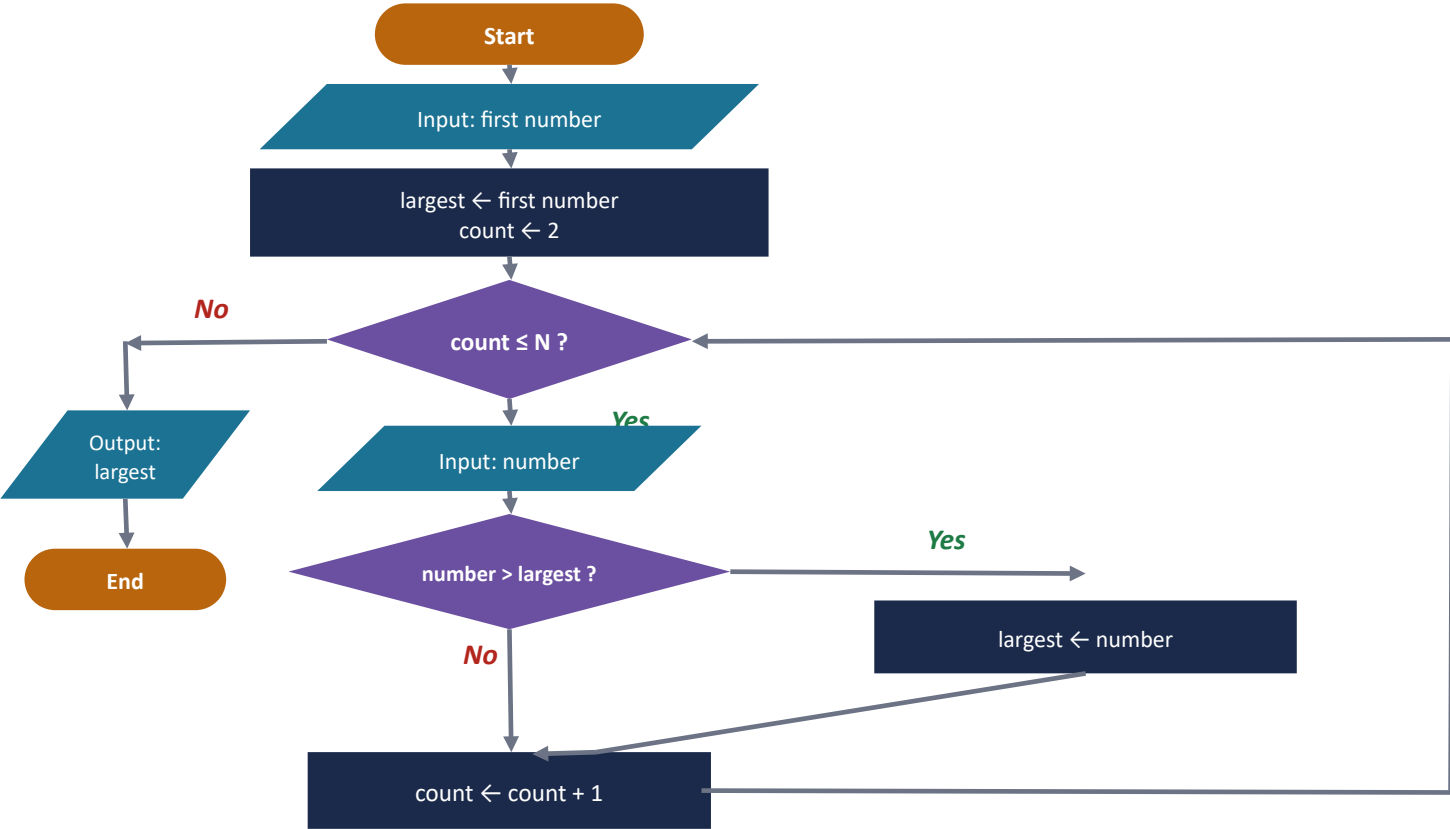


TRACE · N = 4, inputs: 9, 5, 12, 6

i	number	%3=0?	count
1	9	Yes	1
2	5	No	1
3	12	Yes	2
4	6	Yes	3
5	—	exit	output 3

Same shape as sum-positives — only the nested Process box changed from “add” to “count.”

Warm-Up: Track Just One Running Value

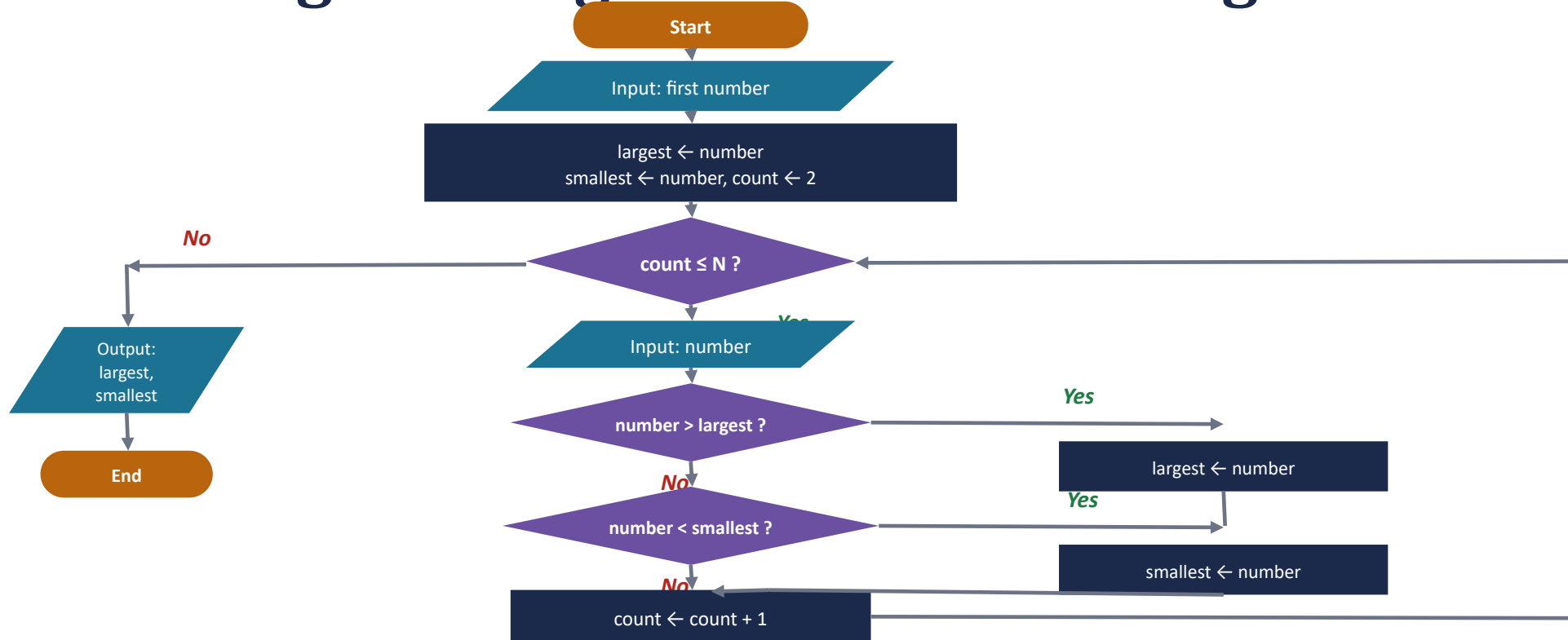


TRACE · first = 8, then 3, 15, 1, 9

count	number	>largest?	largest
—	8 (first)	—	8
2	3	No	8
3	15	Yes	15
4	1	No	15
5	9	No	15
6	—	exit	output 15

Just one variable to track. Next: what if we need two at once?

Tracking the Largest and Smallest Together



TWO IFS, NOT IF/ELSE

A number could update neither tracker. Write two separate decisions — not if/else — so each check runs independently.

Tracing “Max & Min” by Hand

Sample run with N = 5: first number is 3 (so largest = smallest = 3 to start), then 7, -2, 5, 1.

count	number	>largest?	<smallest?	largest	smallest
—	3 (first)	—	—	3	3
2	7	Yes	No	7	3
3	-2	No	Yes	7	-2
4	5	No	No	7	-2
5	1	No	No	7	-2
6	—	exit	exit	output 7	output -2

largest and smallest each change on a different pass — they don't have to move together, or even move at all.

A Loop Inside a Loop

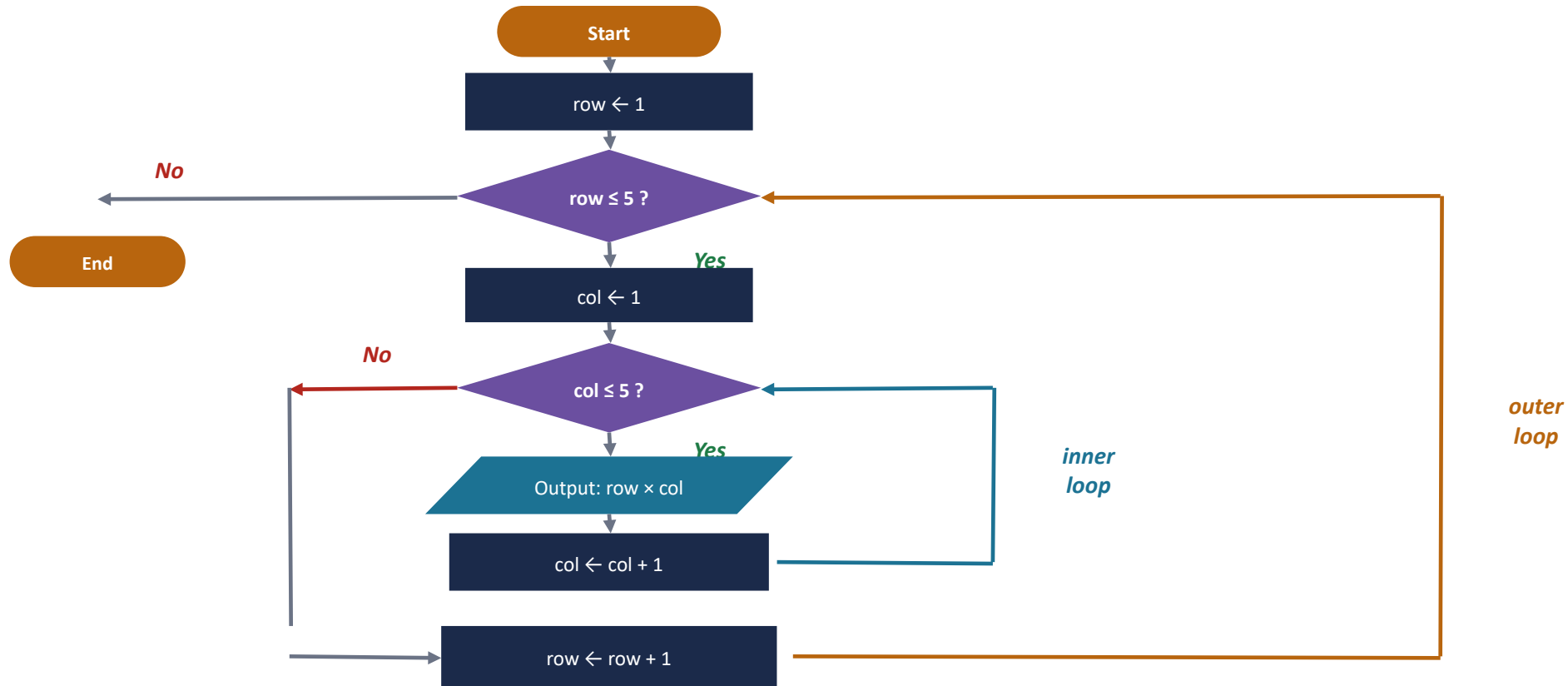
Everything so far has been one loop, sometimes with a decision inside it. Now the thing inside the loop is itself a complete loop — with its own decision and its own backward arrow.

OUTER LOOP — runs once per row

INNER LOOP — runs completely, start to finish, EVERY time the outer loop takes one step

If the outer loop runs 5 times and the inner loop runs 5 times per pass, the innermost step runs $5 \times 5 = 25$ times in total.

Printing a Full Multiplication Grid (1–5)



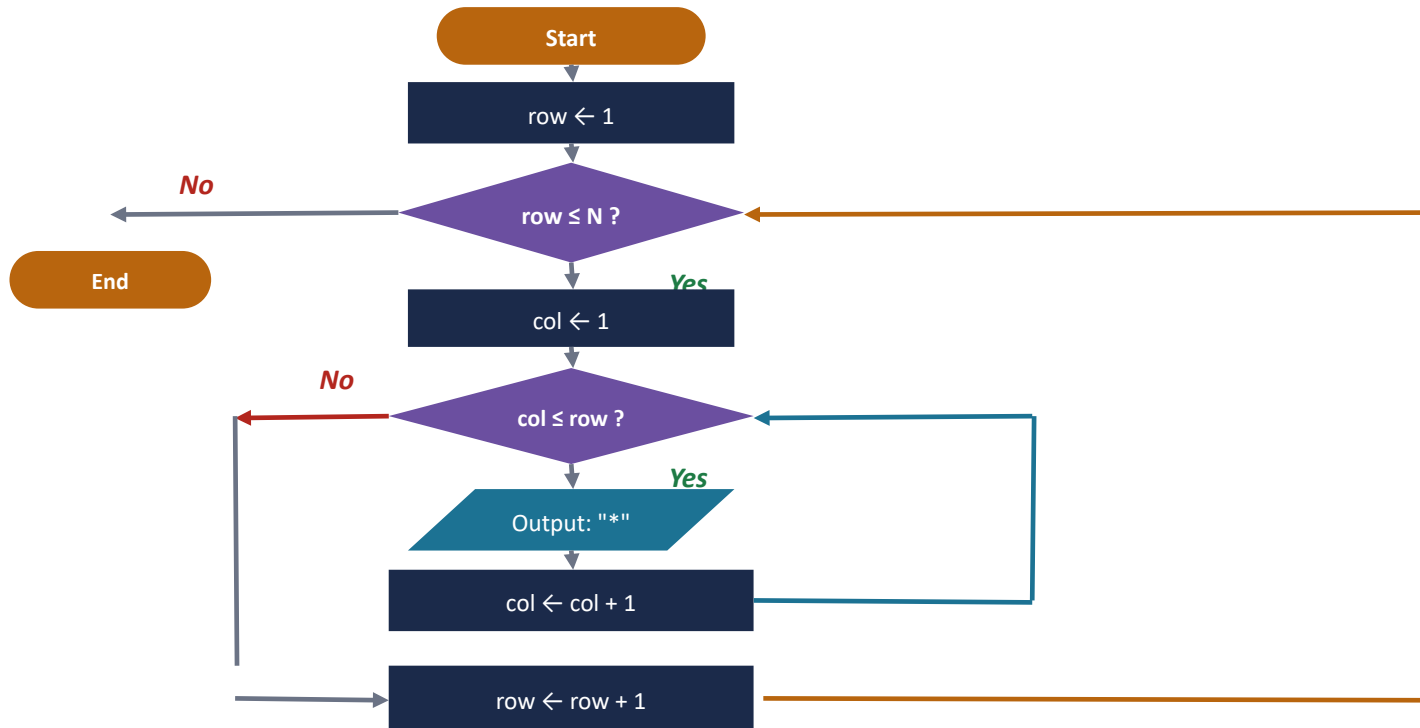
Tracing the Grid: Watch col Reset

The key moment: every time row increases, col goes all the way back to 1.

row	col	output (row × col)
1	1	1
1	2	2
1	3	3
1	4	4
1	5	5
2	1 ← reset!	2
2	2	4
...	...	...
5	5	25 (last)

Inner loop finishes all 5 columns before row ever changes — that's what “the inner loop runs completely” means in practice.

When the Inner Loop's Limit Isn't Fixed

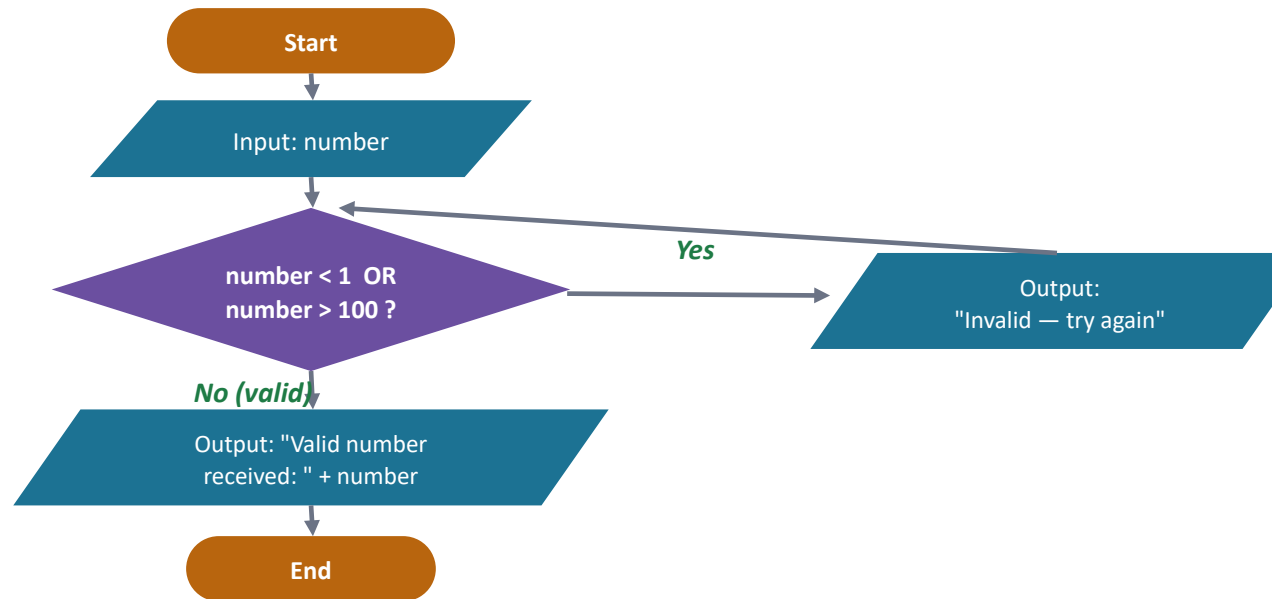


N = 4 produces:

```
*
* *
* * *
* * * *
```

Row 1 stops at col 1. Row 4 stops at col 4. The SAME inner decision ("col ≤ row?") now compares against a value that changes every outer pass.

Keep Asking Until the Value Is Valid



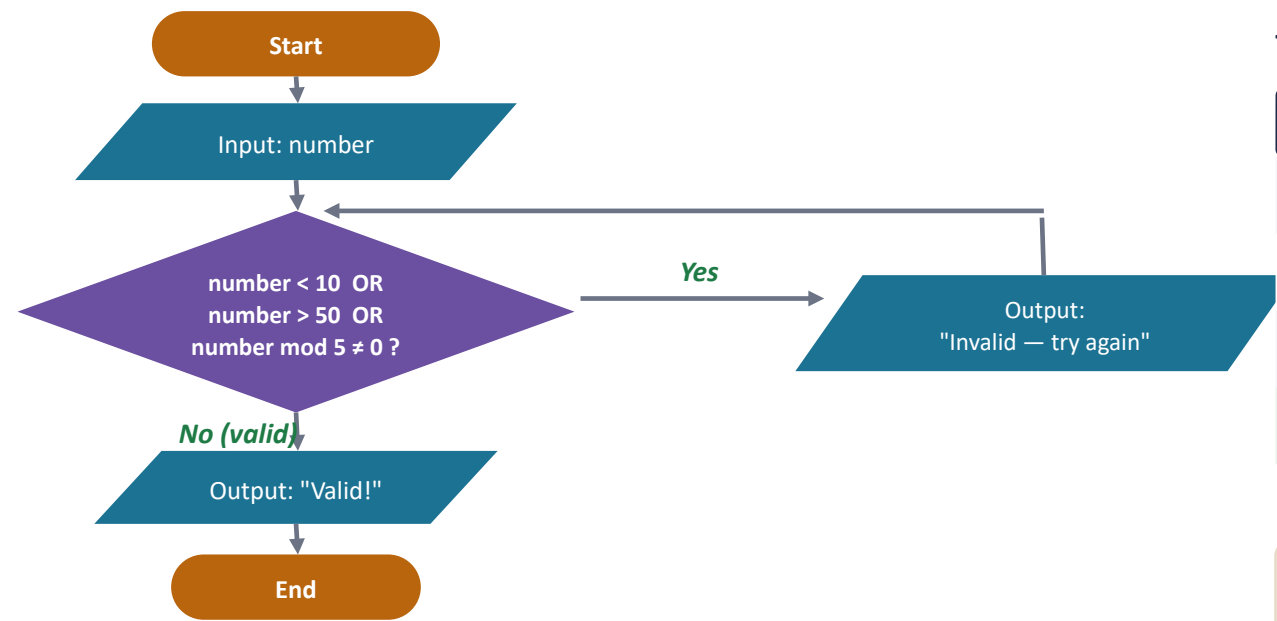
TRACE · inputs: 150, -5, 42

pass	number	out of range?
1	150	Yes — ask again
2	-5	Yes — ask again
3	42	No — accepted!

WHERE THE ARROW GOES

The loop-back returns to Input: number — not to the decision — exactly like last week's sentinel-loop lab question.

One Decision, Three Things to Check



TRACE · inputs: 7, 63, 22, 25

pass	number	why invalid
1	7	< 10
2	63	> 50
3	22	not × 5
4	25	valid!

READING OR CHAINS

Any ONE of the three conditions being true is enough to reject the number — all three must be false for it to be accepted.

A Loop That Can End Two Different Ways

Every loop so far had one reason to stop: the counter ran out. A search stops for either of two reasons — whichever comes first.

Ran out of items

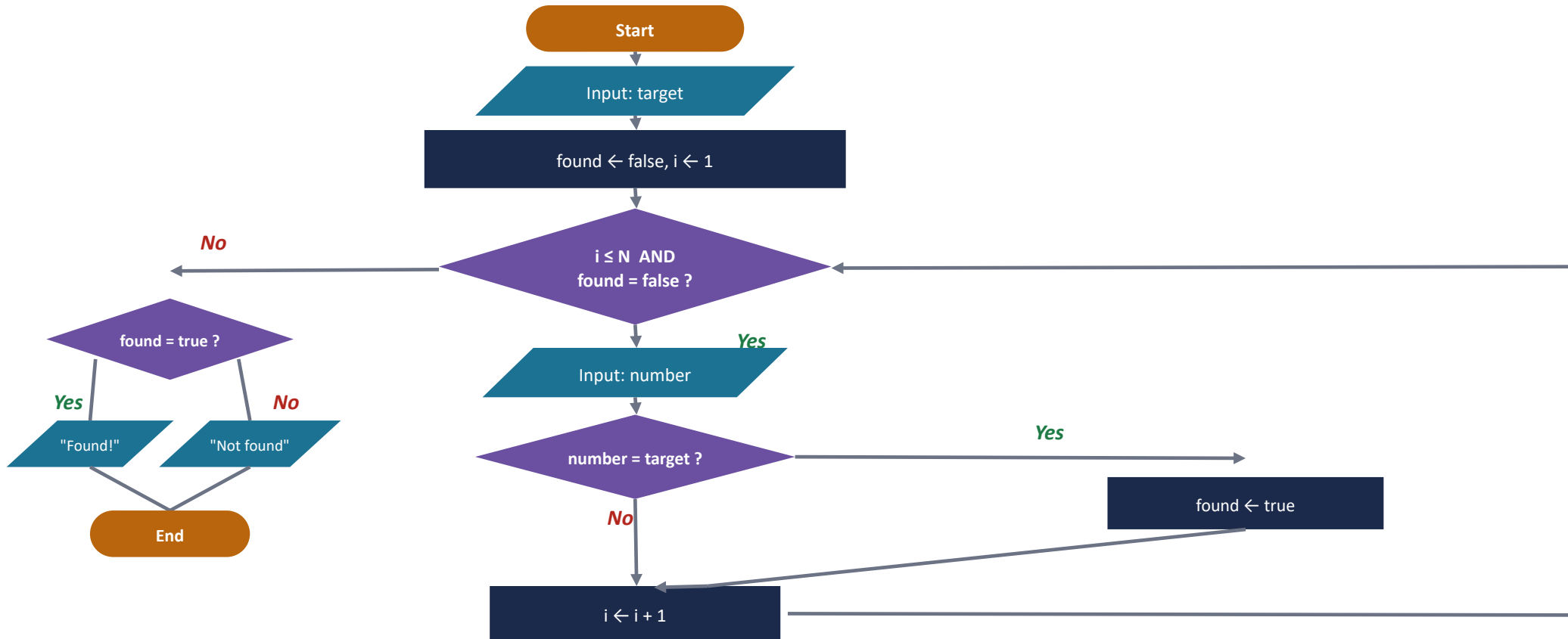
$i > N$ — we checked everyone on the list and never found it.

Found it early

A flag called `found` switches to `true` — no need to check what's left.

The loop's condition becomes: $i \leq N$ AND `found = false` — both must hold for another pass to happen.

Searching a List for a Target Value



Tracing a Search That Stops Early

N = 5, target = 7, inputs in order: 3, 9, 7, 2, 4.

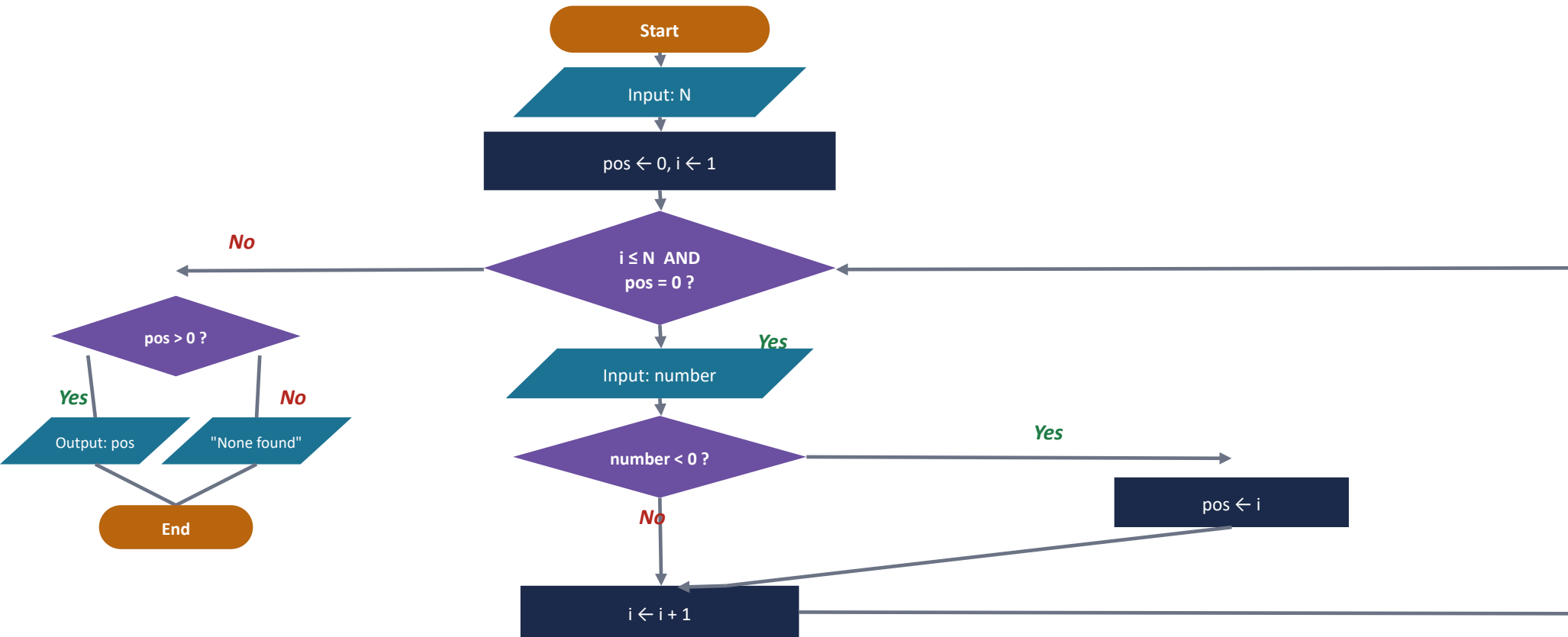
i	i≤N AND !found?	number	number=target?	found
1	Yes	3	No	false
2	Yes	9	No	false
3	Yes	7	Yes	true
4	No — found is true	—	—	exit — output "Found!"

The loop stops after i = 3, even though N = 5 — two numbers (2 and 4) are never even read. That's the whole point of an early stop.

LOOKING AHEAD

In Java, this same early-stop idea is usually written with a break statement inside a simpler while (true) loop — same logic, less bookkeeping.

Finding the Position, Not Just a Yes or No



YES/NO VS. WHERE

Example 1 only needed a found flag. This one needs pos too — one extra variable is often the whole difference between “does it exist” and “where is it.”

Summary & What's Next

- ✓ Conditional accumulation: the loop always updates its counter, but only sometimes updates the total.
- ✓ Tracking two running values (max and min) just means two independent if checks per pass.
- ✓ A loop inside a loop repeats its entire inner loop once for every single pass of the outer one.
- ✓ Input validation loops back to the input step itself, not to the decision that tests it.
- ✓ A search loop can stop two ways — combine both conditions with AND.

Next: we finally put all of this into real Java syntax — variables, if / else, and while / for loops.